

ORIGINAL ARTICLE

Learning unknown reward function for drone navigation based on inverse deep reinforcement learning

Zhe Chen  · Junyu Xuan

Received: 2 May 2025 / Accepted: 12 December 2025 / Published online: 17 February 2026

© The Author(s) 2026

Abstract

Autonomous drone navigation with deep reinforcement learning (DRL) is hindered by the difficulty of specifying reward functions for vision-based, continuous control in complex environments. We address this by using inverse reinforcement learning (IRL) to recover a task-aligned reward directly from expert demonstrations; specifically, we employ adversarial IRL (AIRL) to learn the reward. In evaluation, the learned-reward policy improves success rate, smoothness and trajectories consistency compared with a carefully tuned human-designed reward and baselines reward function. These results indicate that learning the reward from demonstrations provides a precise and transferable objective for autonomous flight, achieving better performance and better guidance under verification of our evaluation protocol without manual reward engineering. To the best of our knowledge, this is the first work to successfully apply an AIRL framework for visual drone navigation.

Keywords Reinforcement learning · Autonomous drone control · Inverse reinforcement learning · Drone navigation

1 Introduction

Drones have become an essential tool across a variety of domains, including surveying and mapping [1], search and rescue [2], agriculture [3, 4], surveillance and conservation of wildlife [5–7], and communication [8]. Due to the wide application of drones and the need for efficient and precise operations in various industries, the demand for drone automatic control is also increasing. Although some traditional strategies were able to achieve automatic control of drones in a specific task and environment, they usually rely on pre-programmed instructions, like the exact map, operations, the starting and ending positions, optimal path, etc. The recent rapid development of reinforcement learning (RL) makes it a more suitable tool for automatic drone control [9–11]. Compared with traditional methods, RL can do autonomous navigation across diverse environments without the need for precise positioning, significantly enhancing robustness in complex environments [12]. Furthermore, RL can manage high-dimensional state spaces, such as those involving depth images and velocity data, and continuous action spaces. This capability is crucial for complex tasks like obstacle avoidance and navigation in intricate environments.



Reinforcement learning is grounded in the theoretical framework of the Markov Decision Process (MDP) to accurately define states, actions, state transition function, and reward function [13]. The design of the reward function for a specific task is crucial for success. However, because of the complexity of the environment, designing an entirely suitable reward function for drone navigation tasks is challenging, such as reward sparsity, inaccuracies in reward functions, and conflicts among multiple objectives [14]. Firstly, a reward signal is typically provided only when a task is completed or a sub-task is completed, resulting in a lack of intermediate step signals. This scarcity slows the learning process and may cause the algorithm to converge to a local optimum. Secondly, in complex environments, the required reward signal guidance for drones is difficult to generalise by simple mathematical models, leading to reward functions that diverge from the actual task objectives. Finally, when performing drone navigation tasks, it is sometimes necessary to consider multiple external factors, which may conflict in practical applications, such as safety, efficiency, energy consumption, etc. At the same time, some external factors are abstract, such as images, which can make it difficult to design rewards. Consequently, balancing these demands through a designed reward function.

To overcome these limitations, we propose to use Inverse Reinforcement Learning (IRL) [15] to infer the unknown reward function from human demonstrations. However, traditional IRL, such as Bayesian IRL and Maximum Entropy IRL, offer a decoupled reward function approach for drones but still face challenges related to insufficient generalisation and an over-reliance on expert demonstrations. These limitations arise from constraints in the feature set and the state space [16, 17]. Consequently, there is a significant impact on drone navigation and continuous control tasks. Hence, we adopt Adversarial Inverse Reinforcement Learning (AIRL), which leverages a GAN-inspired structure within the IRL framework [18]. AIRL is not limited by feature design and state space, and through the adversarial training mechanism, it can decouple a reward function with robustness and stability, and has certain transferability. This algorithm could lead to significant performance breakthroughs in drone navigation tasks. We propose a reinforcement learning framework for drone high-dimensional narrow-space navigation missions that leverages an automatically extracted reward function via AIRL. Our goal is to solve the problem of reward design for drones in high-risk visual spaces and complex continuous action spaces, and ultimately achieve an agent for navigation and obstacle avoidance of drones without manually designing reward functions to train policy.

The main contributions of this paper can be summarised as:

1. We propose an approach to infer the unknown reward function for drone navigation tasks from a few near-expert demonstrations. Our approach is based on Inverse Reinforcement Learning, specifically Adversarial Inverse Reinforcement Learning. The reward function inferred by our approach has been proven to be safer (higher success rate), smoother and more consistent than human-designed reward function level.
2. We applied our approach to drone navigation in a complex and narrow-space obstacle environment, characterised by a high-dimensional local observation space comprising depth images and velocity data and a mainstream realistic drone flight control continuous action space. We validate our approach's applicability and effectiveness in approximate-real-world drone control scenarios through this simulation environment.

The remainder of this paper is organized as follows. In the following section, we discuss the recent related work on the drone navigation task. In Sect. 3, we provide a brief introduction to proximal policy optimization, the reinforcement learning method used in this research. The methodology section focuses on the drone navigation problem and how the reward function can be inferred from near-expert demonstrations. In Sect. 5, we provide a comparative experiment to verify the performance of the reward function from our approach and the humanly crafted one and related experimental results. For reproducibility, our implementation and datasets can be found on our GitHub.¹

¹ Github link and experiment result available at: https://github.com/zitchan/Dronenvai_airl.git

2 Related work

In this section, we focus primarily on the related work of drone navigation algorithms based on local data. For example, drones rely on their sensors for navigation, such as vision, speed, gravity, acceleration, and other sensors.

2.1 Traditional methods

Traditional machine learning drone navigation systems are typically divided into navigation systems based on global maps or not [19]. Among off-map methods, early approaches such as optical flow [20], vision-based texture tracking [21], and vision-based obstacle estimation method [22] have been representative. These methods determine the drone's position by calculating changes in external features. However, the navigation performance achieved by these techniques is often imprecise; as a result, they are no longer considered the mainstream technology for drone navigation and are used primarily as aids for control or attitude stabilization, such as during landing and takeoff [19].

Another approach involves constructing a map, either directly or indirectly through the use of sensors. In this context, Simultaneous Localization and Mapping (SLAM) is predominantly applied to drone navigation and obstacle avoidance. For example, vision-based LSD-SLAM systems have been developed for drone obstacle avoidance and navigation [23]; laser scanning is used to build a 3D point map of the environment, thereby supporting navigation and exploration by delineating boundaries [24]; and hybrid EKF-based inertial navigation systems that combine visual and laser SLAM have also been explored [25].

Although the traditional machine learning method is a direct and fast way to locate and guide the position of the drone, it is insufficient to rely on a large amount of sensor information to ensure its anti-interference. Especially under noise conditions, maps constructed through data streams accumulate under noise and thus affect their performance.

2.2 Imitation learning

Imitation learning (IL) is an algorithm that can be directly and efficiently applied to the automatic control of unmanned aerial vehicles. Different from Reinforcement Learning, it does not require an explicitly designed reward function. It adopts a kind of supervised learning, which learns a decision-making policy by observing and copying expert demonstrations.

Behavior Cloning (BC) is a widely used baseline in drone navigation tasks, where the expert's state-action trajectories are directly cloned to train a policy. Research has shown that BC can complete relatively stable navigation tasks in specific environments and tasks [26–28]. BC is prone to the distribution shift problem: when the agent encounters states that are not fully represented in the training data, the errors will intensify over time, leading to subpar performance. This limitation motivates the development of more robust methods that can handle the problem of bias in the distribution of expert data.

The DAgger algorithm has been applied to the drone navigation task. Through dataset aggregation, it extends the training set with new states and actions collected during policy execution, thus achieving improved robustness and generalization ability [28, 29]. In addition, to further reduce the reliance on static expert data, adversarial imitation learning methods such as Generative Adversarial Imitation Learning (GAIL), which is based on GAN ideas, have been used for drone navigation tasks [30]. GAIL employs an adversarial framework to learn the agent's behavior with expert demonstrations, although it does not explicitly recover reward functions.

Compared to traditional techniques such as Simultaneous Localization and Mapping (SLAM), imitation learning does not require explicit environmental modeling, thereby reducing computational demands and mitigating the adverse effects of sensor noise [26]. However, imitation learning methods remain heavily dependent on

high-quality expert data, which can lead to issues such as error accumulation, persistent distribution shift, and insufficient exploration. More importantly, it performs poorly outside the distribution of expert data.

2.3 Deep reinforcement learning

Deep Reinforcement Learning (DRL) has become a powerful paradigm for drone navigation, enabling both discrete and continuous control across diverse environments. A growing body of work shows that DRL can learn effective navigation policies: promising results have been reported in discrete-control settings [31–33] and in more challenging continuous-control tasks [9, 34, 35], highlighting the potential of end-to-end learning with high-dimensional perception for autonomous flight.

Nevertheless, current DRL systems still fall short of human-level reliability. In essence, DRL—whether policy-gradient, value-based, or actor–critic—optimizes behavior under the guidance of a reward signal [36]. In practice, the reward must often be iteratively adjusted to meet task expectations, and its design is non-trivial: practitioners confront sparsity, misspecification, and multi-objective trade-offs (e.g., balancing goal reaching, safety, and smoothness) [14, 37]. These issues interact with exploration, so that even with improved shaping, performance can remain constrained by how data are collected and by safety considerations during learning. Moreover, trajectory quality may lag behind classical planners—e.g., producing less smooth paths—under comparable conditions [34]. Because outcomes hinge on how well the reward captures task semantics, constraints, and preferences, small specification errors can propagate into brittle policies, increase tuning cost, and limit generalization across layouts or controller interfaces. These considerations motivate a complementary line of inquiry that focuses on principled reward design and reward extraction for drone navigation, aiming to reduce manual engineering while preserving reliability and transfer.

2.4 Inverse reinforcement learning

Unlike traditional DRL, IRL provides a solution for agent learning and training by demonstrating substitutes manual specification of reward. It can make some behaviors and strategies that are difficult to explain or mathematically model concrete through the demonstrations of experts [38]. Since its introduction by Ng and Russell [15], Inverse Reinforcement Learning (IRL) has substantial interest across robotics, autonomous control and human modeling.

In the IRL application domain, robotic control has a tangible impact. Representative task is the work from Abbeel et al. [39] which realized helicopter aerobatics. This work shows that IRL can be used to infer low-level control. Following this milestone, it has been extended to include robot control scenarios, including motion and manipulation [40–42].

Admittedly, IRL has been widely used in robotics control, but rarely attempted on drones. Possibly limited by the complexity of drone continuous control and the high-dimensional observation space (visually based). Within the drone task, several representative IRL alternatives are nontrivial to instantiate at scale. MaxEnt IRL and its deep/guided variants typically require feature engineering or state–action discretization together with approximate soft value iteration, which is brittle and computationally heavy in this regime [43–45]. Preference-based reward learning, in turn, introduces human-in-the-loop labeling cost and consistency issues for long-horizon flight [46]. As a result, the reward functions of most drone navigation via reinforcement learning algorithms still rely on manual design, which still requires a lot of trial and adjustment for most tasks. These limitations make IRL particularly appealing for drones. Instead of hand-crafting dense shaping terms for goal reaching, collision avoidance, and smoothness under partial observability, IRL seeks a task-aligned reward from expert trajectories, thereby reducing trial-and-error tuning, manual engineering and mitigating reward misspecification. These provides a clear motivation to investigate IRL for drone navigation.

3 Preliminary knowledge

Proximal Policy Optimisation (PPO) [47] is a widely used reinforcement learning algorithm. Due to its simple deployment and excellent stability, it is used in mechanical control, drone control, and other fields. It mainly consists of two neural networks, respectively the policy network and the critic network, which are optimised for training through the Actor-Critic structure. It is widely used in reinforcement learning because of its excellent performance and stability. The flow of the algorithm is shown in Fig. 1.

Firstly, the policy network π_ψ will interact with the environment to collect the trajectories τ_π and store them in the Rollout Buffer (here in AIRL training, it will share the buffer with AIRL). Then, given the collected trajectory, the empirical return $\hat{R}_t$ and the advantage estimate $\hat{A}_t$ for each timestep t are computed. The advantage is estimated using Generalized Advantage Estimation (GAE) as follows:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots \quad (1)$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

where γ is the discount factor, and λ is a bias-variance trade-off parameter. With advantage estimates in hand, the policy network π_ψ is updated by maximizing the clipped surrogate objective:

$$L^{\text{CLIP}}(\psi) = E_t \left[\min \left(r_t(\psi) \hat{A}_t, \text{clip}(r_t(\psi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3)$$

$$r_t(\psi) = \frac{\pi_\psi(a_t|s_t)}{\pi_{\psi_{\text{old}}}(a_t|s_t)} \quad (4)$$

where $r_t(\psi)$ is the probability ratio between the new and old policies. In parallel, the value function $V_\omega(s)$ is updated by minimizing the squared error between the predicted value and the empirical return:

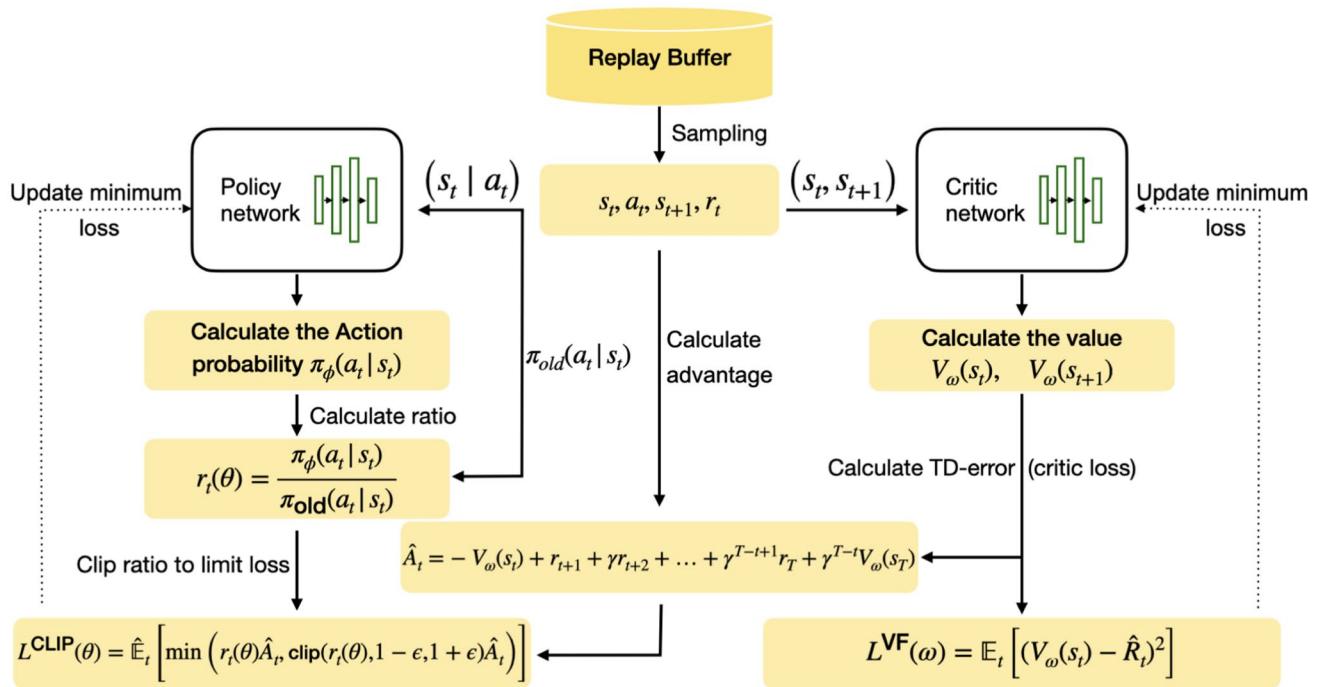


Fig. 1 Flowchart of the algorithmic operation of PPO

$$L^{\text{VF}}(\omega) = E_t \left[\left(V_\omega(s_t) - \hat{R}_t \right)^2 \right]. \quad (5)$$

Through continuous iteration, the algorithm can obtain an optimized policy network. The pseudo-code of the algorithm is specifically shown in Algorithm 1.

Require: Initial policy π_ψ , value function V_ω , environment E , clip ratio ϵ , learning rate α , number of epochs K

1: **for** iteration = 1 to N **do**

2: Collect trajectories

$$\tau_\pi = \{(s_t, a_t, r_t, s_{t+1})\}$$

 by running π_ψ in environment E

3: Compute advantage estimates $\hat{A}_t$ using GAE(λ)

4: Compute old policy probabilities $\pi_{\psi_{\text{old}}}(a_t | s_t)$

5: **for** epoch = 1 to K **do**

6: Compute surrogate objective:

$$L^{\text{CLIP}}(\psi) = E_t \left[\min \left(r_t(\psi) \hat{A}_t, \text{clip}(r_t(\psi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

7: Update policy parameters

$$\psi \leftarrow \psi + \alpha \nabla_\psi L^{\text{CLIP}}(\psi)$$

8: Update value function parameters ω by minimizing:

$$L^{\text{VF}}(\omega) = E_t \left[\left(V_\omega(s_t) - \hat{R}_t \right)^2 \right]$$

9: **end for**

10: **end for**

11: **return** Optimized policy π_ψ

Algorithm 1 Proximal policy optimization (PPO)

Adversarial Inverse Reinforcement Learning (AIRL) [18] recovers a reward function from expert demonstrations in an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma)$ by framing IRL as a two-player game between a discriminator D_θ and a policy π . Given tuples (s, a, s') sampled from the expert π_E and the current policy π , the discriminator is parameterized in a reward-potential form:

$$D_\theta(s, a, s') = \frac{\exp(f_\theta(s, a, s'))}{\exp(f_\theta(s, a, s')) + \pi(a | s)}, \quad (6)$$

$$f_\theta(s, a, s') = r_\theta(s, a) + \gamma \Phi_\theta(s') - \Phi_\theta(s), \quad (7)$$

where $r_\theta(s, a)$ denotes the learned state-action reward and Φ_θ is a potential (shaping) function. The training objective is

$$\min_{\pi} \max_{\theta} \mathbb{E}_{(s, a, s') \sim \pi_E} [\log D_\theta(s, a, s')] + \mathbb{E}_{(s, a, s') \sim \pi} [\log(1 - D_\theta(s, a, s'))], \quad (8)$$

while the policy is optimized by reinforcement learning using the learned reward $r_\theta(s, a)$ (the original implementation employed TRPO with an entropy bonus [18]). This potential-based decomposition renders the recovered reward identifiable up to shaping and, in practice, improves transfer relative to using raw discriminator scores.

4 Methodology

We consider a three-dimensional environment where the drone has access to a limited set of local inputs and must navigate through obstacles to reach the end of a predefined route. In training the navigation agent, our objective is to minimize the reliance on manual design so as to enable continuous control of the drone in complex environments. For input selection, we aim to reduce dependence on multiple onboard sensors while adopting mainstream action control strategies, thereby ensuring the reliability of the navigation system and facilitating future deployment on real drones. Moreover, adopting a continuous action space allows for more fine-grained control compared to a discrete action space, enabling the drone to perform smoother and more precise maneuvers. This design also lays the foundation for future transfer to real-world drone applications. Consequently, we propose an image-based reinforcement learning framework with continuous action control.

In this study, we employ Adversarial Inverse Reinforcement Learning (AIRL) [18], combined with Proximal Policy Optimization (PPO) [47], to jointly learn an unknown reward function and an agent policy. These methods are carefully adapted to the drone navigation setting by defining state and action spaces, as well as network architectures, that are tailored to high-dimensional visual inputs and continuous control requirements.

4.1 Problem formulation and solution

We formulate the vision-based drone navigation task with continuous control as a Markov Decision Process (MDP), which provides a formal framework for training in reinforcement learning. This framework is widely adopted to describe sequential decision-making problems under uncertainty. An MDP is typically defined as a quintuple:

$$M = (S, A, P, r, \gamma), \quad (9)$$

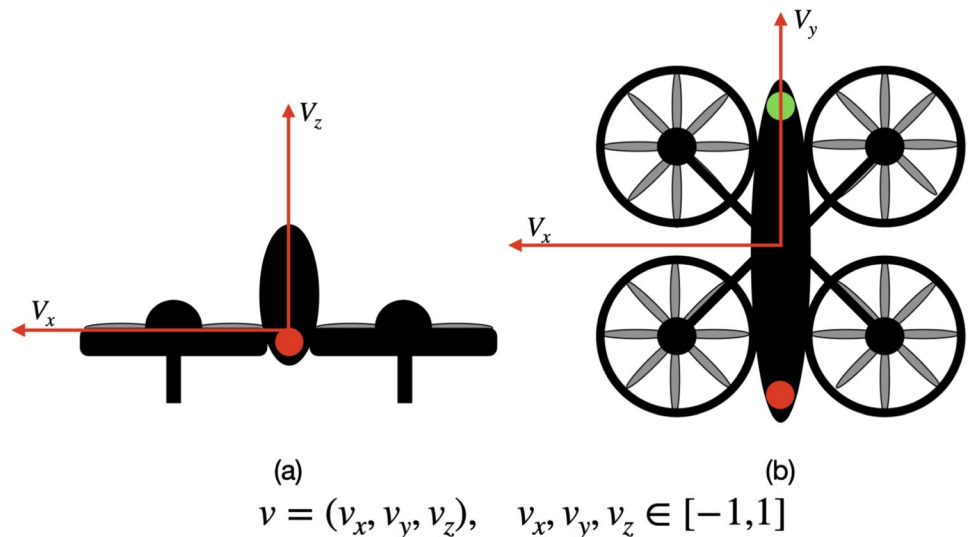
where each element is defined as follows:

- State space S :
 - A single-frame depth image $I \in R^{72 \times 128}$, captured by the onboard camera, encoding the spatial structure of the surrounding environment and normalized to the range $[0, 1]$;
 - A 3-dimensional linear velocity vector $v \in R^3$, representing the drone's instantaneous motion along the x , y , and z axes.
- Action space A : According to the current mainstream input methods of flight control systems (PX4 Offboard / ArduPilot GUIDED), we define a continuous action space, where each action $a \in R^3$ corresponds to a normalized target velocity vector $a = (a_x, a_y, a_z)$ with components in the range $[-1, 1]$. These values are mapped to actual velocities as follows:
 - a_x and a_z are linearly mapped to velocities in the range $[-1, 1]$ unit/s for the x - and z -axes;
 - a_y is mapped to $[-0.5, 1.5]$ unit/s along the y -axis.

These low-level control signals are interpreted by the drone's flight controller and directly determine the drone's movement in three-dimensional space. The actual direction of action of the drone is shown in Fig. 2. Where this action space is mapped by $[1.5v_x, 1.5v_y + 0.5, 1.5v_z]$,

- Transition dynamics $P(s'|s, a)$: The transition function models the environment's stochastic dynamics, representing the probability of transitioning to state s' after taking action a in state s . In our setting, this is realized by a high-fidelity drone simulator (e.g., AirSim), which incorporates physics-based motion models, sensor

Fig. 2 Action space of drone: these are two views of the drone, the left image shows the back view of the drone and the right image shows the top view of the drone



noise, and environment interactions. By adding a bias to v_y , the actual action of the drone will have a forward preference

- Discount factor $\gamma \in [0, 1]$: The discount factor controls the trade-off between immediate and future rewards. A higher γ encourages long-term planning behavior. Meaning that drones focus on long-term goals, such as reaching the final destination.
- Reward function $r(s, a)$: In traditional drone navigation tasks, the reward function relies on manual design. It directly determines the performance of the agent and the success rate of task completion. Drones in high-dimensional complex environments are often difficult to model mathematically, especially when the inputs are abstract and missing global data.

Therefore, our research question is how to achieve drone navigation in a high-dimensional narrow space without using a reward function learning in an unknown environment and guiding the agent to learn a drone navigation strategy with inputs of local data (images and velocity only).

4.2 Learning unknown reward functions for drone navigation tasks

Our method is based on Adversarial Inverse Reinforcement Learning (AIRL), originally proposed by Fu et al. [18], to address the challenging issue of reward function design for drone navigation in high-dimensional complex environments. Different from traditional reinforcement learning methods, which typically rely on human-designed rewards that are sensitive to environmental changes, it automatically infers robust and generalizable reward signals directly from expert demonstrations. When the drone is in an environment where global data is not available, the drone can interactively assess and learn from the environment through expert trajectories.

Fu et al. [18] proposed two versions of AIRL: state-only AIRL and full state-action AIRL. The main difference between the two lies in whether the action space is introduced during discriminator updates for training with unknown rewards. While the full state-action method demonstrates superior performance in OpenAI Gym tasks, it struggles with dynamics and environmental transfer, and loses some interpretability. We adopt the state-only variant of AIRL because, in drone navigation, the unknown reward should encode the task itself (e.g., progress toward the goal, collision avoidance, and safety margins) than replicate the demonstrator's action patterns. Learning $r(s)$ aligns the recovered reward with outcome-level semantics that are invariant across controllers and action parameterizations, enabling portability when moving from simulation to hardware or when switching from velocity to attitude control. In contrast, $r(s, a)$ tends to entangle task evaluation with the demonstrator's stylistic preferences and may degrade under action-space or dynamics changes. Therefore, we favor state-only AIRL: it

captures what constitutes successful navigation, not how an expert happened to actuate it, yielding rewards that generalize and transfer more reliably across drone platforms and environments. Given the complex and dynamic nature of drone environments and task characteristics, this paper applies state-only AIRL.

As illustrated in Fig. 3, we manually collect the expert demonstrations through the simulated environment, denoted by τ_E . It combines a state with 72×128 depth image and a three-direction velocity vector, and a three-direction action vector, which offers guidance of unknown reward training. In contrast, trajectories generated by the agent (policy network), denoted by τ_π , are collected through interaction with the environment. The reinforcement learning algorithm PPO applied in this experiment is to output a distribution of actions, which consists of a mean and a variance, which needs to be mapped through a normal distribution before it is the final vector of actions performed, so that the exploration of the drone in an unknown area can be controlled by the variance.

The reward function of the drone navigation task is adversarially learning by the generator and the discriminator (reward estimation network). Optimisation of the drone navigation strategy and the learning of an unknown reward function dependent on the current state and action are achieved by reducing the error in the trajectory and expert distribution of the agent. In the reward learning process for the navigation, the reward estimation network accepts depth images $I \in R^{72 \times 128}$ and velocity vectors $v \in R^3$ as inputs, as does the policy network, and the output is a value used to guide the agent to the optimal strategy.

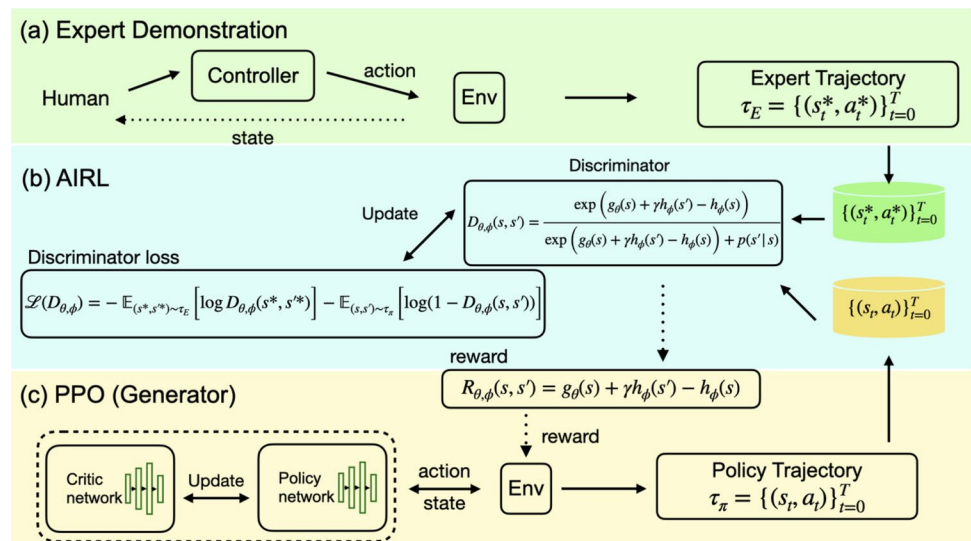
The discriminator network $D_{\theta,\phi}(s, s')$ differentiates state transitions sampled from expert drone demonstrations and those generated by the current drone agent. It consists of two key components: an immediate reward function $g_\theta(s)$ and a state potential function $h_\phi(s)$. Formally, the discriminator is defined as:

$$D_{\theta,\phi}(s, s') = \frac{\exp(g_\theta(s) + \gamma h_\phi(s') - h_\phi(s))}{\exp(g_\theta(s) + \gamma h_\phi(s') - h_\phi(s)) + p(s'|s)}, \quad (10)$$

where $g_\theta(s)$ represents the immediate reward at state s , which means offering a quality closing degree between the current depth image and velocity vectors of the drone and the expert distribution. $h_\phi(s') - h_\phi(s)$ denotes the potential (value estimation) of state s and next state s' , γ is the discount factor controlling the importance of future states, which is a hyperparameter for balancing short-term and long-term incentives. It affects the actual flight efficiency and flight success of drones in navigation missions, and $p(s'|s)$ is the state-transition distribution determined by the drone's policy, which refers to the transfer probability of the combination of depth image and velocity of the drone before taking the action and after taking action.

Using this structure, it explicitly decouples the reward function as:

Fig. 3 Flowchart of AIRL-based drone navigation training framework



$$R_{\theta,\phi}(s, s') = g_{\theta}(s) + \gamma h_{\phi}(s') - h_{\phi}(s), \quad (11)$$

where the immediate reward component $R_{\theta,\phi}(s, s')$ quantifies the intrinsic quality or desirability of the current drone depth images $I \in R^{72 \times 128}$ and velocity vectors $v \in R^3$ and the next depth images $I' \in R^{72 \times 128}$ and velocity vectors $v' \in R^3$. The potential-based shaping term $\gamma h_{\phi}(s') - h_{\phi}(s)$ represents the change in long-term navigational value upon transitioning from the current state s to the next state s' . A positive potential difference implies a favorable transition, encouraging the drone's flight policy towards long-term beneficial states, while negative values discourage transitions to undesirable states.

The drone policy (generator) in this paper is trained using Proximal Policy Optimization (PPO), detailed in Sect. 3.

The unknown reward function training involves following iterative steps:

1. Expert demonstrations collecting: Expert trajectories τ_E are collected by the human interacting with the environment.
2. Trajectory generation: Drone trajectories τ_{π} are generated by the current policy interacting with the environment.
3. Discriminator update: The discriminator network is updated to distinguish between expert trajectories and generated drone trajectories. This update involves minimizing the discriminator loss:

$$L(D_{\theta,\phi}) = -E_{(s^*, s'^*) \sim \tau_E} [\log D_{\theta,\phi}(s^*, s'^*)] - E_{(s, s') \sim \tau_{\pi}} [\log(1 - D_{\theta,\phi}(s, s'))]. \quad (12)$$

4. Generator (Policy) update: Using the reward provided by the discriminator (Eq. 11), the drone policy parameters ψ are updated via PPO (Sect. 3), progressively enhancing its ability to replicate expert-level navigation behavior.

The details of the unknown reward function training loop are summarized in Algorithm 2. By iterating through these adversarial training steps, the drone policy converges toward behaviors indistinguishable from expert demonstrations, ultimately obtaining a robust and transferable navigation policy suitable for complex approximate-real-world scenarios.

Require: Expert trajectories τ_E

- 1: Initialize drone policy $\pi_{\psi}(a|s)$ and discriminator $D_{\theta,\phi}(s, s')$
- 2: **for** iteration = 1 to N **do**
- 3: Generate drone trajectories τ_{π} using policy $\pi_{\psi}(a|s)$
- 4: Update discriminator (θ, ϕ) by maximizing:

$$E_{(s^*, s'^*) \sim \tau_E} [\log D_{\theta,\phi}(s^*, s'^*)] + E_{(s, s') \sim \tau_{\pi}} [\log(1 - D_{\theta,\phi}(s, s'))]$$

- 5: Compute shaped reward: $R_{\theta,\phi}(s, s') = g_{\theta}(s) + \gamma h_{\phi}(s') - h_{\phi}(s)$
 - 6: Update drone policy parameters ψ via PPO using reward $R_{\theta,\phi}(s, s')$
 - 7: **end for**
 - 8: **return** Unknown reward function $R_{\theta,\phi}(s, s')$
-

Algorithm 2 Unknown reward function learning for drone navigation

5 Experimental setting

² Networks and discriminator architecture. We initialize four neural networks: a policy π_{ψ} for action generation, a value function V_{ω} for state-value estimation, and an AIRL discriminator with two heads, a reward head g_{θ} and a potential head h_{ϕ} . As shown in Fig. 4, the perception module uses a six-layer convolutional encoder to extract and

² The reward network and the potential network in the discriminator remain the same structure and independent networks.

Fig. 4 Illustration of the network structure of a vision-based navigation neural network for the drone

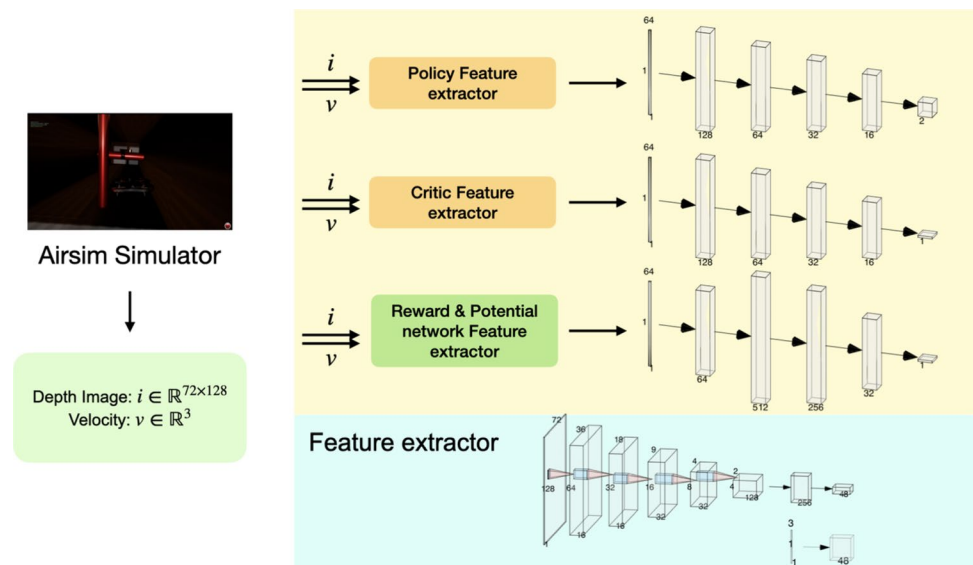
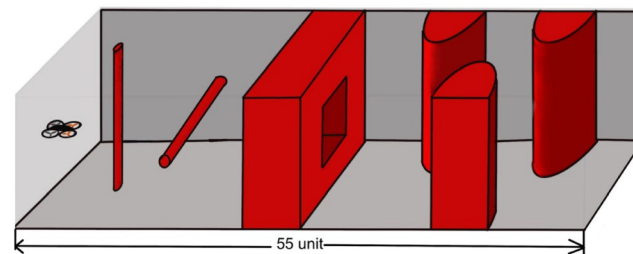


Fig. 5 The drone interaction environment, based on unreal engine design for the environment



downsample features from the depth image; the velocity vector is processed by a lightweight MLP and rescaled to a comparable magnitude. The two streams are concatenated into a unified state representation. This shared representation feeds the policy and value networks and also serves as the common encoder for the discriminator, from which the reward head g_θ and the potential head h_ϕ branch as shallow MLPs with linear outputs. The two heads have the same network architecture (same number of layers, hidden widths, and activations) but separate, non-shared parameters. This fusion design balances the contributions of depth and velocity and reduces feature-importance bias in the learned representation.

5.1 Environment design

In this experiment, as Fig. 6 shows, we used AirSim as a simulator, a platform for experimental research on drones based on Unreal Engine 4.27, developed by Microsoft Research. It has complete and simulated physical interaction and built-in sensors [48]. We deployed an obstacle tunnel, and the specific obstacles are shown in the Fig. 5.³ It mainly includes three parts: two bar obstacles, a window obstacle, and a set of S-shaped obstacles. The drone needs to fly 55 units of length to reach the endpoint. During this period, the drone needs to change its height and accurately avoid obstacles left and right to reach the target area.

³ Reference environment from <https://github.com/sunghoonhong/AirsimDRL>

of 55 units along the task axis Y within at most 300 actions and without collision, which reflects the *effectiveness* of the learned policy; the average flight distance during training reflects the reward's *guidance capability* for reinforcement learning. Formally, letting stage $m \in \{1, \dots, M\}$ contain B evaluation trajectories with sampled positions $\{p_t^{(m,i)} = (x_t^{(m,i)}, y_t^{(m,i)}, z_t^{(m,i)})\}_{t=0}^{T_{m,i}}$ (where $T_{m,i} \leq 300$) and $\text{coll}_{m,i} \in \{0, 1\}$ indicating collision (0 means collision-free), we define

$$\text{SR}^{(m)} = \frac{1}{B} \sum_{i=1}^B s_{m,i}, \quad (13)$$

$$\overline{D}^{(m)} = \frac{1}{B} \sum_{i=1}^B (y_{T_{m,i}}^{(m,i)} - y_0^{(m,i)}). \quad (14)$$

For smoothness on *successful* test trajectories (lower is better), speed-domain smoothness is measured by RMS Jerk: with sampling interval Δt ,

$$v_t = \frac{p_{t+1} - p_t}{\Delta t}, \quad a_t = \frac{v_{t+1} - v_t}{\Delta t}, \quad j_t = \frac{a_{t+1} - a_t}{\Delta t} \quad (15)$$

$$\text{RMS_Jerk} = \left(\frac{1}{N_j} \sum_t \|j_t\|_2^2 \right)^{1/2}, \quad (16)$$

capturing the average magnitude of changes in acceleration (smaller values indicate smoother control with fewer abrupt corrections) [49]. Geometry-domain smoothness is measured by the heading-rate RMS: letting the unit tangent be $\hat{t}_t = \frac{p_{t+1} - p_t}{\|p_{t+1} - p_t\|_2}$ and the turning angle $\theta_t = \arccos(\hat{t}_t^\top \hat{t}_{t+1})$, we compute

$$\text{RMS_HeadRate} = \left(\frac{1}{N_\theta} \sum_t \left(\frac{\theta_t}{\Delta t} \right)^2 \right)^{1/2}, \quad (17)$$

which reflects turning smoothness (smaller values imply gentler turns) [50]. Trajectory consistency is quantified by the *pairwise lateral RMSE*: aligning successful trajectories by progress Y on a grid $\{y_k\}_{k=1}^K$ via linear interpolation to obtain $(x_i(y_k), z_i(y_k))$, for any pair (i, j) we define

$$d_{ij}(y_k) = \sqrt{(x_i(y_k) - x_j(y_k))^2 + (z_i(y_k) - z_j(y_k))^2}, \quad (18)$$

$$\text{RMSE}_{ij} = \sqrt{\frac{1}{|K_{ij}|} \sum_{k \in K_{ij}} d_{ij}(y_k)^2}, \quad (19)$$

and report the mean across all pairs,

$$\text{Mean Pairwise RMSE} = \frac{2}{N_s(N_s-1)} \sum_{i < j} \text{RMSE}_{ij}, \quad (20)$$

where N_s is the number of successful trajectories; smaller values indicate more concentrated and repeatable paths (more stable policy) [51].

5.3 Experiments conducted

As shown in Fig. 7, the experimental workflow comprises two stages: (i) a preprocessing phase and (ii) learning of the unknown reward function. The preprocessing phase includes AirSim environment setup, acquisition of expert demonstrations, and policy network preprocessing. Learning the unknown reward function is the core component of our method. We then conduct evaluation to quantify how different reward formulations influence PPO training performance.

To obtain expert trajectories in a continuous action space, we adopt a keyboard-to-velocity mapping; the discrete base actions are listed in Table 2. Because the drone requires continuous inputs, we inject zero-mean continuous

Table 2 Manual drone control actions and corresponding AirSim velocity vectors

Direction	AirSim action (velocity vector) ¹
Forward	[0, 0.8, 0]
Backward	[0, -0.8, 0]
Left	[0.8, 0, 0]
Right	[-0.8, 0, 0]
Ascend	[0, 0, 0.8]
Descend	[0, 0, -0.8]
Forward-Left	[0.8, 0.8, 0]
Forward-Right	[-0.8, 0.8, 0]
Backward-Left	[0.8, -0.8, 0]
Backward-Right	[-0.8, -0.8, 0]
Ascend-Forward	[0, 0.8, 0.8]
Ascend-Backward	[0, -0.8, 0.8]
Ascend-Left	[0.8, 0, 0.8]
Ascend-Right	[-0.8, 0, 0.8]
Descend-Forward	[0, 0.8, -0.8]
Descend-Backward	[0, -0.8, -0.8]
Descend-Left	[0.8, 0, -0.8]
Descend-Right	[-0.8, 0, -0.8]

¹ The action here refers to the action before mapping, the actual execution of the action will be processed by the environment

Table 3 Success/failure counts and smoothness/consistency (success-only metrics): our method vs human-designed

Metric	Our method	Human-designed reward
Successes (n)	24	21
Failures (n)	6	9
Success rate	0.8	0.7
RMS jerk	0.2835	0.403
RMS HeadRate	0.5887	0.5416
Mean pairwise RMSE	0.4963	0.6529

noise of ± 0.2 into each action component to emulate continuous control. For example, the forward action is $a_t = [0, 0.8, 0]$ (bound to the w key), which in a specific context is executed as $a_t = [0.15423, 0.86461, 0.13523]$. This procedure converts operator inputs into continuous actions for logging. Although it introduces stochasticity, in practice we retain only expert trajectories that successfully traverse obstacles, yielding a simple and low-overhead way to collect high-quality continuous-action demonstrations (Table 3).

Given the resource demands of adversarial training, we first pretrain the drone policy via behavior cloning to shorten wall-clock time and ensure sufficient generator trajectory lengths; the reward network remains the primary computational bottleneck. In total, we conduct approximately 9.75 million environment steps of training (Table 4).

For the navigation environment, we ran multiple PPO configurations under human-designed rewards to obtain a stable training setup. The final hyperparameters used in subsequent PPO training are summarized in Table 5. Among them, the initial log standard deviation plays a pivotal role by setting the initial exploration level so that the drone can fly stably while still exploring; this also prevents exploration from confounding comparisons across reward variants. In Sect. 5.4, the performance of human-designed rewards reflects extensive tuning of reward magnitudes, forms, and combinations.

All experiments were conducted on a workstation with an NVIDIA GeForce RTX 2070 Super GPU (8 GB) and 16 GB of system RAM. The simulation environment used AirSim 1.5.0 with the Unreal Engine 4.27 physics engine. The obstacle map was constructed from UE4 *Block* assets; the detailed obstacle layout is shown in Fig. 6.

Table 4 Full reproducibility details

Item	Value	Notes
<i>Hardware</i>		
GPU	NVIDIA GeForce RTX 2070 Super (8 GB)	Single-GPU training
System RAM	16 GB	–
CPU	Intel Core i7-10700K (16 M Cache, up to 5.10 GHz)	–
OS	Windows 10	64-bit
<i>Software</i>		
Visual studio	2022	–
Python	$\geq 3.10.13$	Exact version
AirSim	1.5.0	Client + UE plugin
Unreal engine	4.27	Physics engine
AirSim map base	Block	UE4 <i>Blocks</i> assets
Gymnasium	$\geq 0.29.1$	–
PyTorch	$\geq 2.6.0$	–
Imitation	1.0.1	–
clock speed	1	–
n_env	1	–
Seed	345	Fix across Python/NumPy/PyTorch
<i>AIRL training settings</i>		
demo_batch_size	512	–
gen_replay_buffer_capacity	2048	–
n_disc_updates_per_round	4	–
disc_opt_kwargs	{“lr”: 1e-4, “weight_decay”: 1e-4}	Adam optimizer

Table 5 Hyperparameters and network configuration for PPO training

Parameter	Value
<i>Policy network</i>	
Policy network architecture (pi)	[128, 64, 32, 16]
Value network architecture (qf)	[128, 64, 32, 16]
Shared feature extractor	False
Initial log standard deviation	−1.8
<i>Training settings</i>	
GAE lambda (λ)	0.95
Entropy coefficient	0.01
Learning rate	5×10^{-4}
Number of steps per rollout	2048
Batch size	2048
Discount factor (γ)	0.99
Clip range	0.2
Advantage normalization	True
Value function coefficient	0.5
Gradient clipping norm	2
Number of epochs	64
Random seed	345

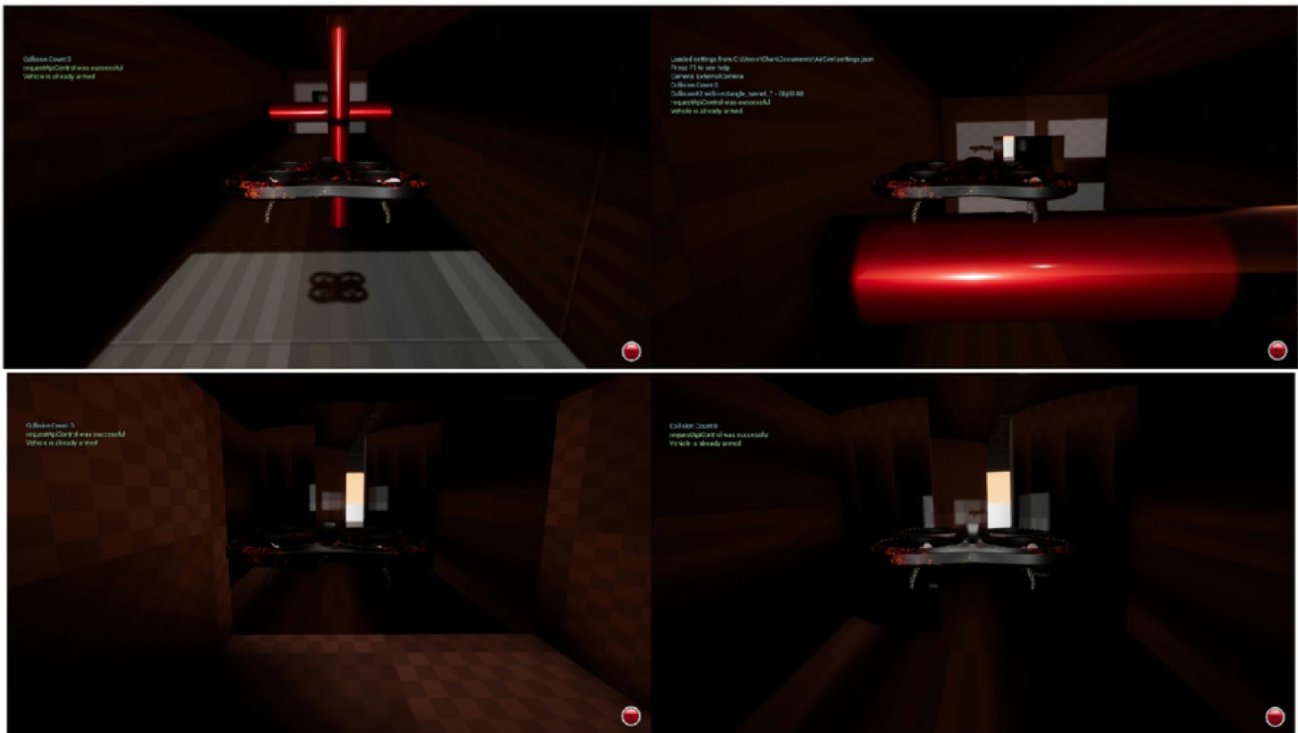


Fig. 6 The screenshot of the AirSim simulator

5.4 Experiments result

⁴⁵ As illustrated in the Figs. 9 and 10, we evaluated three distinct reward functions for training reinforcement learning models in a drone navigation task. The first strategy employed a reward function learned via our method to train the PPO agent, the second utilized the strong human-designed reward function baseline; and the third adopted a sparse reward function; and the last adapted a simple dense reward function.

Comparative experimental results demonstrated that the reward function derived through AIRL significantly enhanced both the success rate and average flight distance of the agent. Specifically, after approximately 150,000 training steps, the AIRL-based reward function consistently maintained a success rate of around 80%. In contrast, the PPO agent trained with the manually designed dense reward exhibited an overall rising trend during training but experienced considerable fluctuations and instability, achieving a success rate varying between 40% to 70% and an inconsistent mean flight distance. Meanwhile, the agent trained with the sparse reward and the simple dense reward proved inadequate for the drone navigation task, consistently failing to reach the goal and showing minimal improvement in flight distance. Globally, although the agent trained by the strong human-designed reward function baseline can reach the destination earlier, the success rate of the agent trained by the reward function learned by our method at the beginning of the training of about 10,000 steps is exceeded and most of the time is much higher than the strong human-designed reward function baseline.

From the average distance flown, we can see that different reward functions perform differently in guiding the training of the agent. Between 35 and 45 units is mainly the interval of S-shaped obstacles. Between 60,000 and 100,000 steps, our method allows the agent to learn a strategy to pass the S-shaped obstacle steadily, whereas

⁴ 30 tests followed each 10,000 training iterations, simple dense reward and sparse reward success rate is always 0%

⁵ Each 10,000 training iterations is followed by 30 tests. The mean distance is the average flight distance in 30 tests.

Fig. 8 Comparison of 3D trajectory sets

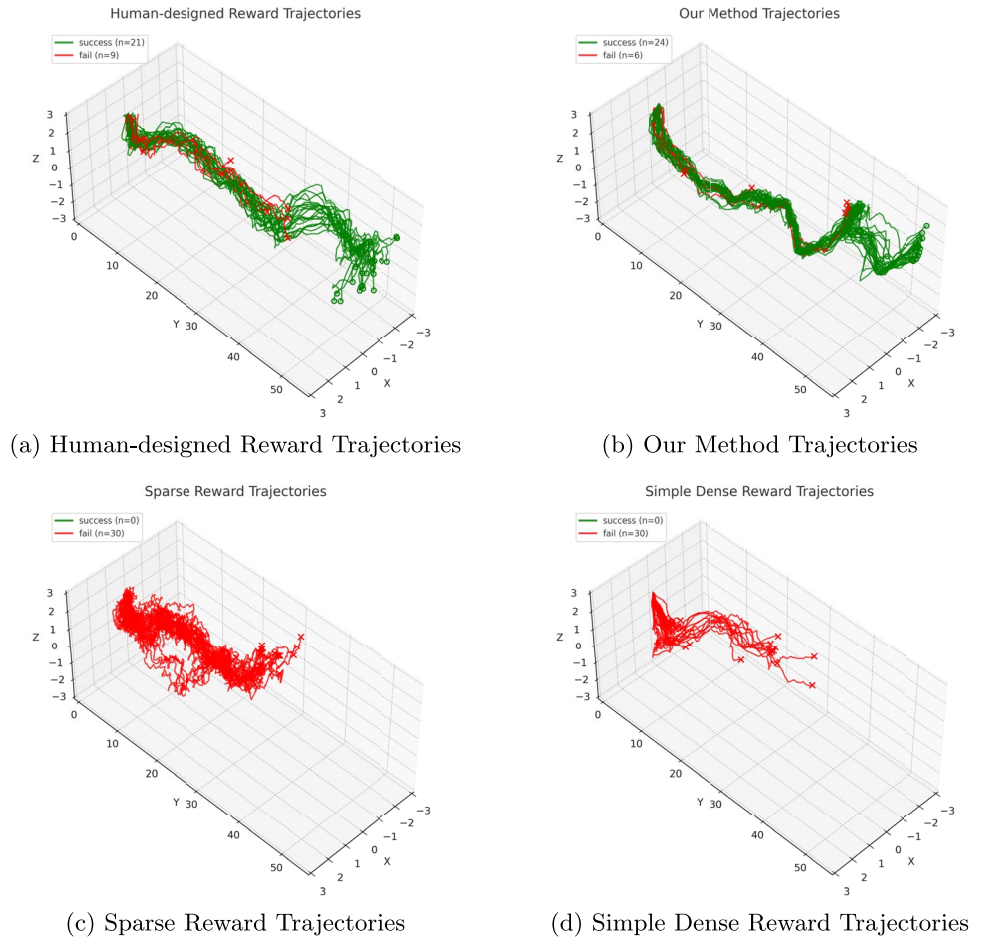


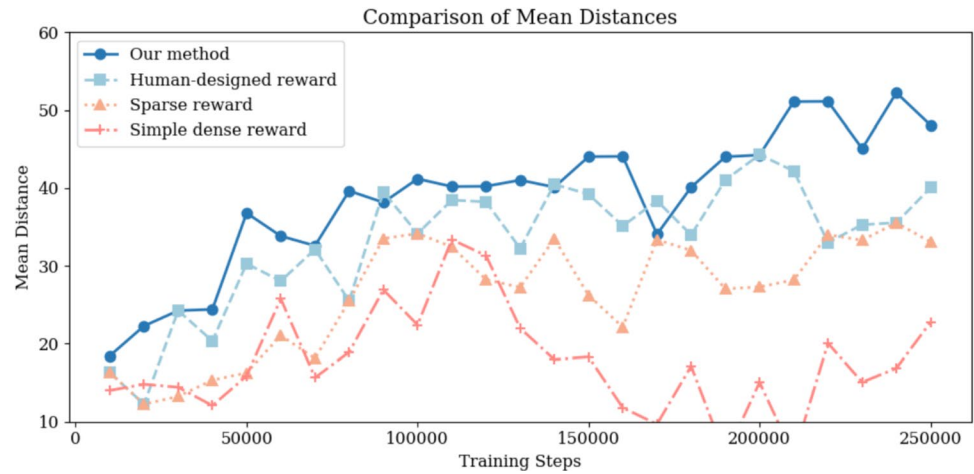
Fig. 9 Success rate comparison across different reward strategies



the strong human-designed reward function baseline requires more steps from 40,000 to 120,000, and the agent trained by the Sparse reward function is almost unable to learn a strategy to pass the obstacle in that region.

As illustrated in the Fig. 8, only our method and the strong human-designed baseline reward function can learn a valid policy. The flight trajectories of the agents trained by the sparse reward function mostly collide around 30 units, which means that it has not yet learned the strategy for passing through S-shaped obstacles. The collision points of the agents trained by Simple dense reward are more concentrated at 15 to 20 units, which means

Fig. 10 Mean distance comparison across different reward strategies



it is almost difficult for it to learn a stable flight strategy. According to the Table 3, we can see that the speed smoothness (RMS Jerk) of our method is smoother compared to the method of the strong human-designed reward baseline, but the geometric smoothness (RMS HeadRate) is close to that of human-designed reward. This means that our method has less variation in flight speed, less energy consumption and less jitter of the drone, and closed turning amplitude. As for the degree of trajectory overlap, the trajectory consistency (Mean PairWise RMSE) of the trajectory learned by our method is even better.

6 Limitation and future work

The current simulation does not explicitly model disturbances such as wind, precipitation, fog, illumination, texture shifts, sensor noise, or communication impairments—which limits external validity in complex outdoor settings. All experiments were conducted in a virtual environment without hardware-in-the-loop or real-flight validation, so platform constraints remain untested. In addition, the on-policy training pipeline requires frequent collection of fresh rollouts, leading to lower sample efficiency and higher computational cost, which remains a common challenge for traditional on-policy reinforcement learning methods [52].

We will broaden environmental realism and test, then validate on hardware to establish external validity. Methodologically, we aim to improve sample efficiency and training stability with more data-efficient RL and better reuse of experience, while maintaining safety and interpretability. Also representation learning with lightweight, compute-aware network structure (e.g., MobileNetV3, MnasNet, EfficientNetV2) [53–55] and self-contrastive regularization (e.g., DrQ, CURL, SPR, SVEA) is a potential way to enhance efficiency [56–59]. These directions are intended to enable scalable deployment across layouts, platforms, and control interfaces.

Beyond these extensions, we will also explore integrating large language models (LLMs) into the autonomy stack as high-level semantic priors for intent understanding, task structuring, and constraint-aware planning. The goal is to let language-grounded abstractions organize perception–action loops and guide the use of experience so that efficiency, safety, and interpretability can be improved jointly [60].

7 Conclusion

This study explored the efficacy of AIRL in addressing drone navigation tasks in complex environments, with a special focus on overcoming the challenges of reward design in RL. Comparative experiments were conducted that our method enables the learning of an unknown reward function that effectively guides reinforcement

learning agents in drone navigation tasks. Through verification, the rewards obtained by our method show that it has better adaptability to continuous actions and visual-based observation space drone navigation tasks, as well as reinforcement learning guidance capabilities. Consequently, it addresses the challenging issue of manually designing reward functions for drone navigation, especially within complex and uncertain environments.

Appendix

For reproducibility, our implementation and datasets can be found on our github link at: https://github.com/zitchan/Dronenvai_airl.git

Acknowledgements This work is supported by the Australian Research Council under the Discovery Early Career Researcher Award DE200100245

Author contributions Zhe Chen and Junyu Xuan jointly contributed to Conceptualization and Methodology. Zhe Chen was responsible for Software, Validation, Formal Analysis, Investigation, Visualization, and Writing—Original Draft. Junyu Xuan was responsible for Project Administration, Supervision, Writing—Review & Editing, and Funding Acquisition. Both authors read and approved the final manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions

Data availability All code and experiment result available at: https://github.com/zitchan/Dronenvai_airl.git. This study does not involve any other publicly available datasets.

Declarations

Conflict of interest Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ri S, Fujigaki M, Morimoto Y (2010) Sampling moiré method for accurate small deformation distribution measurement. *Exp Mech* 50(4):501–508. <https://doi.org/10.1007/s11340-009-9239-4>
2. Gonzalez V, Jaillet P (2025) Multi-drone rescue search in a large network. *Eur J Oper Res*. <https://doi.org/10.1016/j.ejor.2025.02.003>
3. Peña JM, Torres-Sánchez J, Serrano-Pérez A, De Castro AI, López-Granados F (2015) Quantifying efficacy and limits of unmanned aerial vehicle (uav) technology for weed seedling detection as affected by sensor resolution. *Sensors* 15(3):5609–5626. <https://doi.org/10.3390/s150305609>
4. Peña JM, Torres-Sánchez J, Serrano-Pérez A, De Castro AI, López-Granados F (2024) Smart agriculture drone for crop spraying using image-processing and machine learning techniques: experimental validation. *IoT* 5(2):250–270
5. Gonzalez LF, Montes GA, Puig E, Johnson S, Mengersen K, Gaston KJ (2016) Unmanned aerial vehicles (uavs) and artificial intelligence revolutionizing wildlife monitoring and conservation. *Sensors* 16(1):97. <https://doi.org/10.3390/s16010097>
6. Coluccia A, Fascista A, Sommer L, Schumann A, Dimou A, Zarpalas D, Sharma N (2023) Drone-vs-bird detection grand challenge at icassp2023. In: *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–2. <https://doi.org/10.1109/ICASSP49357.2023.10433921>

7. Sharma N, Saqib M, Scully-Power P, Blumenstein M (2022) In: Chen, F., Zhou, J. (eds.) *SharkSpotter: Shark Detection with Drones for Human Safety and Environmental Protection*, pp. 223–237
8. Yanmaz E, Yahyanejad S, Rinner B, Hellwagner H (2018) Bettstetter C (2018) Drone networks: communications, coordination, and sensing. *Ad Hoc Netw* 68:1–15. <https://doi.org/10.1016/j.adhoc.2017.09.001>
9. Hodge VJ, Hawkins R, Alexander R (2021) Deep reinforcement learning for drone navigation using sensor data. *Neural Comput Appl* 33(6):2015–2033
10. Kaufmann E, Bauersfeld L, Loquercio A, Müller M, Koltun V, Scaramuzza D (2023) Champion-level drone racing using deep reinforcement learning. *Nature* 620(7976):982–987. <https://doi.org/10.1038/s41586-023-06419-4>
11. Yu H, Wager C, Croon GCHE (2025) Mavrl: learn to fly in cluttered environments with varying speed. *IEEE Robot Autom Lett* 10(2):1441–1448. <https://doi.org/10.1109/LRA.2024.3522778>
12. Aburaya A, Selamat H, Muslim MT (2024) Review of vision-based reinforcement learning for drone navigation. *Int J Intell Robot Appl* 8(4):974–992. <https://doi.org/10.1007/s41315-024-00356-9>
13. Sutton RS, Barto AG (2018) *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, Massachusetts
14. Chan JH, Liu K, Chen Y, Sagar ASMS, Kim Y-G (2024) Reinforcement learning-based drone simulators: survey, practice, and challenge. *Artif Intell Rev* 57(10):281. <https://doi.org/10.1007/s10462-024-10933-w>
15. Ng AY, Russell S (2000) Algorithms for inverse reinforcement learning. In: *ICML*, p. 2
16. Ramachandran D, Amir E (2007) Bayesian inverse reinforcement learning. In: *IJCAI*, pp. 2586–2591
17. Ziebart BD, Maas AL, Bagnell JA, Dey AK (2008) Maximum entropy inverse reinforcement learning. In: *Aaai*, pp. 1433–1438
18. Fu J, Luo K, Levine S (2017) Learning robust rewards with adversarial inverse reinforcement learning. *arXiv preprint arXiv:1710.11248*
19. Lu Y, Xue Z, Xia G-S, Zhang L (2018) A survey on vision-based uav navigation. *Geo-spat Inf Sci* 21(1):21–32
20. Herissé B, Hamel T, Mahony R, Russotto F-X (2012) Landing a vtol unmanned aerial vehicle on a moving platform using optical flow. *IEEE Trans Rob* 28(1):77–89. <https://doi.org/10.1109/TRO.2011.2163435>
21. Li H, Yang SX (2003) A behavior-based mobile robot with a visual landmark-recognition system. *IEEE/ASME Trans Mechatron* 8(3):390–400
22. Javid A, Alnaser M, Baroudi U, Alfaraj A (2025) Monocular vision navigation system for uav autonomous mission: a real-time window-based obstacle avoidance approach. *Neural Comput Appl*. <https://doi.org/10.1007/s00521-025-11093-w>
23. Stumberg L, Usenko V, Engel J, Stückler J, Cremers D (2017) From monocular slam to autonomous drone exploration. In: *2017 European Conference on Mobile Robots (ECMR)*, pp. 1–8
24. Tiemann J, Ramsey A, Wietfeld C (2018) Enhanced uav indoor navigation through slam-augmented uwb localization. In: *2018 IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1–6. <https://doi.org/10.1109/ICCW.2018.8403539>
25. Magree D, Johnson EN (2014) Combined laser and vision-aided inertial navigation for an indoor unmanned aerial vehicle. In: *2014 American Control Conference*, pp. 1900–1905. <https://doi.org/10.1109/ACC.2014.6858995>
26. Candare RJ, Dagui RC (2020) Deep imitation learning for safe indoor autonomous micro aerial vehicle navigation. In: *2020 IEEE 12th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*, pp. 1–6. <https://doi.org/10.1109/HNICEM51456.2020.9399994>
27. Fan Y, Chu S, Zhang W, Song R, Li Y (2020) Learn by observation: Imitation learning for drone patrolling from videos of a human navigator. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5209–5216. <https://doi.org/10.1109/IROS45743.2020.9340691>
28. Loquercio A, Kaufmann E, Ranftl R, Müller M, Koltun V, Scaramuzza D (2021) Learning high-speed flight in the wild. *Sci Robot* 6(59):5810
29. Amer K, Samy M, Shaker M, Elhelw M, Osten W, Zhou J, Nikolaev DP (2021) Deep convolutional neural network based autonomous drone navigation. In: *Thirteenth International Conference on Machine Vision*
30. Bandela S, Cao Y (2023) Drone navigation in unreal engine using generative adversarial imitation learning. In: *AIAA SCITECH 2023 Forum*, p. 0506
31. Çetin E, Barrado C, Muñoz G, Macias M, Pastor E (2019) Drone navigation and avoidance of obstacles through deep reinforcement learning. In: *2019 IEEE/AIAA 38th Digital Avionics Systems Conference (DASC)*, pp. 1–7. <https://doi.org/10.1109/DASC43569.2019.9081749>
32. Shin S-Y, Kang Y-W, Kim Y-G (2019) Obstacle avoidance drone by deep reinforcement learning and its racing with human pilot. *Appl Sci* 9(24):5571
33. Fu C, Xu X, Zhang Y, Lyu Y, Xia Y, Zhou Z, Wu W (2022) Memory-enhanced deep reinforcement learning for uav navigation in 3d environment. *Neural Comput Appl* 34(17):14599–14607. <https://doi.org/10.1007/s00521-022-07244-y>
34. Chronis C, Anagnostopoulos G, Politi E, Dimitrakopoulos G, Varlamis I (2023) Dynamic navigation in unconstrained environments using reinforcement learning algorithms. *IEEE Access* 11:117984–118001. <https://doi.org/10.1109/ACCESS.2023.3326435>

35. Song Y, Steinweg M, Kaufmann E, Scaramuzza D (2021) Autonomous drone racing with deep reinforcement learning. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 1205–1212
36. Kheddar H, Dawoud DW, Awad AI, Himeur Y, Khan MK (2024) Reinforcement-learning-based intrusion detection in communication networks: a review. *IEEE Commun Surv Tutor*. <https://doi.org/10.1109/COMST.2024.3484491>
37. Wang C, Wang J, Wang J, Zhang X (2020) Deep-reinforcement-learning-based autonomous uav navigation with sparse rewards. *IEEE Internet Things J* 7(7):6180–6190
38. Arora S, Doshi P (2021) A survey of inverse reinforcement learning: challenges, methods and progress. *Artif Intell* 297:103500. <https://doi.org/10.1016/j.artint.2021.103500>
39. Abbeel P, Coates A, Quigley M, Ng A (2006) An application of reinforcement learning to aerobatic helicopter flight. In: *Advances in Neural Information Processing Systems*
40. Johannink T, Bahl S, Nair A, Luo J, Kumar A, Loskyll M, Ojea JA, Solowjow E, Levine S (2019) Residual reinforcement learning for robot control. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 6023–6029
41. Vollenweider E, Bjelonic M, Klemm V, Rudin N, Lee J, Hutter M (2023) Advanced skills through multiple adversarial motion priors in reinforcement learning. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 5120–5126
42. Zhang X, Sun L, Kuang Z, Tomizuka M (2021) Learning variable impedance control via inverse reinforcement learning for force-related tasks. *IEEE Robot Autom Lett* 6(2):2225–2232
43. Ziebart BD, Maas A, Bagnell JA, Dey AK (2008) Maximum entropy inverse reinforcement learning. In: *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438
44. Wulfmeier M, Ondruška P, Posner I (2015) Maximum entropy deep inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*
45. Finn C, Levine S, Abbeel P (2016) Guided cost learning: Deep inverse optimal control via policy optimization. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pp. 49–58
46. Christiano PF, Leike J, Brown TB, Martic M, Legg S, Amodei D (2017) Deep reinforcement learning from human preferences. In: *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4299–4307
47. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*
48. Shah S, Dey D, Lovett C, Kapoor A (2017) Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In: *Field and Service Robotics*
49. Hogan N, Sternad D (2009) Sensitivity of smoothness measures to movement duration, amplitude, and arrests. *J Mot Behav* 41(6):529–534
50. De Novellis L, Sorniotti A, Gruber P (2015) Driving modes for designing the cornering response of fully electric vehicles with multiple motors. *Mech Syst Signal Process* 64–65:1–15. <https://doi.org/10.1016/j.ymssp.2015.03.024>
51. Sturm J, Engelhard N, Endres F, Burgard W, Cremers D (2012) A benchmark for the evaluation of RGB-D SLAM systems. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vilamoura, Algarve, Portugal, pp. 573–580. <https://doi.org/10.1109/IROS.2012.6385773>
52. Deisenroth MP, Neumann G, Peters J (2013) A survey on policy search for robotics. *Found Trends Robot* 2(1–2):1–142
53. Howard A, Sandler M, Chu G, Chen L-C, Chen B, Tan M, Wang W, Zhu Y, Pang R, Vasudevan V, Le QV, Adam H (2019) Searching for mobilenetv3. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324
54. Tan M, Chen B, Pang R, Vasudevan V, Sandler M, Howard A, Le QV (2019) Mnasnet: Platform-aware neural architecture search for mobile. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2820–2828
55. Tan M, Le Q (2021) Efficientnetv2: Smaller models and faster training. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. *Proceedings of Machine Learning Research*, vol. 139, pp. 10096–10106. PMLR, Virtual Event (Online)
56. Yarats D, Kostrikov I, Fergus R (2021) Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In: *International Conference on Learning Representations (ICLR)*
57. Laskin M, Srinivas A, Abbeel P (2020) Curl: Contrastive unsupervised representations for reinforcement learning. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. *Proceedings of Machine Learning Research*, vol. 119, pp. 5639–5650. PMLR, Virtual Event (Online)
58. Schwarzer M, Anand A, Goel R, Hjelm RD, Courville A, Bachman P (2021) Data-efficient reinforcement learning with self-predictive representations. In: *International Conference on Learning Representations (ICLR)*
59. Hansen N, Su H, Wang X (2021) Stabilizing deep q-learning with convnets and vision transformers under data augmentation. In: *Advances in Neural Information Processing Systems (NeurIPS)*
60. Kheddar H, Habchi Y, Ghanem MC, Hemis M, Niyato D (2025) Recent Advances in Transformer and Large Language Models for UAV Applications (2025). <https://doi.org/10.48550/arXiv.2508.11834>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Zhe Chen¹  · **Junyu Xuan¹**

✉ Zhe Chen
zhe.chen-3@student.uts.edu.au

✉ Junyu Xuan
junyu.xuan@uts.edu.au

¹ The Australian Artificial Intelligence Institute (AAIL), University Technology of Sydney, 61 Broadway, Ultimo, Sydney, NSW 2007, Australia